RFC: Database Schema Design

Project 64: YardWise

Savhanna Beene
beenes@oregonstate.edu

Field	Value
Author	Savhanna
Date	2026-10-24
Status	Proposed
Decision Date	(fill when decided)
Deciders	(who agreed to the outcome)
Superseded By	(link, if applicable)

Problem and Context

YardWise helps homeowners and renters plan, organize, and maintain outdoor projects by tracking plant information, project tasks, budgets, yard photos, and maintenance reminders. An important decision is how YardWise should organize its database schema. It needs to manage information about users, properties, landscaping projects, tasks, materials, and project expenses. For example, one property may have multiple landscaping projects, and each project may contain multiple tasks and materials. The database needs to maintain these relationships while supporting future features and projects.

This is a hard-to-reverse decision because changing table structures and relationships after users enter data may require complicated migrations, updates, and conversions. Poor initial design could lead to duplicate information, inaccurate budgets, missing records, or security problems. Changing database schemas can be time-consuming and may negatively affect existing projects.

We are planning the schema before implementing the main project management features. If the team does nothing, development can continue temporarily, but the database may contain inconsistencies, creating additional integration work later.

Options

Option A: Fully Structured Relational Schema

Design the core tables, relationships, constraints, and validation rules before implementation.

Benefits: PostgreSQL can reject invalid relationships and values before they become stored records. This makes the database easier to understand because each table has a defined responsibility.

Costs: Disadvantages include more planning, more structured migrations, and coordinating schema changes across the team. Adding new relationships or changing required fields may mean updating multiple parts of the application.

Option B: Incremental Relational Schema

Begin with essential tables and introduce additional relationships and constraints as features are developed. Instead of designing all tables and relationships upfront, the team would begin with essential tables such as profiles, properties, and projects. Additional tables for tasks, materials, and expenses would be introduced when those features are ready for implementation.

Benefits: The main advantage is that the team would spend less time designing database structures for features that may change. Prisma migrations could be used to introduce new tables and relationships as the application grows.

Costs: Adding relationships or constraints later could require modifying existing records and code. The team would also need to coordinate migrations carefully to avoid inconsistencies.

Option C: Do Nothing

If we do nothing, the team would continue development without finalizing the database schema. This could cost the team time and resources throughout the project, and communication could get confusing. There may be conflicting expectations about what team members should build.

Benefits: The team could focus on other features for now and avoid spending time on database details that may change.

Costs: The team may overlook requirements or find them more expensive to integrate later. Different members could make conflicting assumptions about how data should be stored. Conflicting code and files could negatively affect documentation and the project workflow. Fixing these issues may require migrations and rework, which could delay completion.

Analysis

Criteria	Option A: Fully Structured	Option B: Incremental	Option C: Do Nothing
Data Integrity	Defines relationships and constraints early to prevent invalid or duplicate data.	Adds relationships and constraints as features are developed. Existing data may need changes later.	Could lead to inconsistent or duplicate information without an agreed structure.
Complexity and Maintainability	Requires more planning upfront but gives the team clear structure.	Allows smaller changes throughout development but may require updating existing code.	Saves planning time now but could create confusion and additional work later.
Performance	Allows the team to plan indexes and queries early to improve data retrieval [1].	Allows the team to improve queries and add indexes as features are developed.	Performance issues may not be discovered until later in development.
Security	Allows the team to plan user permissions and Row Level Security (RLS) early [2].	Security rules must be added and tested as new tables are created.	Security requirements may remain unclear, increasing the risk of mistakes.
Cost	Requires more time at the beginning but may reduce future problems.	Spreads work throughout the project but may require additional changes.	Saves time initially but could result in more work and higher costs later.
Migration Risk	Allows the team to plan and test database changes before implementing major features.	Requires migrations as new features are developed, which could affect existing data.	Delaying the schema decision could require larger database changes later.

Recommendation and Verification Plan

I would select Option A. Although it would require more planning upfront, it would provide a clear structure for implementing the database. This would give the team a better understanding of the project's overall structure and what still needs to be done. The team would know that it

worked by testing the database structure, security, and data retrieval. I would test the relationships between properties, projects, tasks, and materials to make sure they connect correctly. I would also try adding a task to a project that does not exist to make sure the database rejects it. For security, I would use two test accounts to make sure users cannot access each other's private information. I would also test that budgets and costs are stored correctly and that invalid status values are rejected. If any tests fail, we would fix the problems and test again.

Constraints

Licensing: PostgreSQL uses an open-source license. I verified that the official Prisma and Supabase GitHub repositories use the Apache License 2.0. We need to make sure we follow the rules when using outside tools, code, and other resources.

Data use and security: YardWise may store personal information, property details, yard photos, and budgets. We should use sample data when testing and avoid uploading private information to public repositories. Database passwords and private keys should also be kept secure.

Intellectual property and confidentiality: Our team should not share private information in GitHub or AI tools without permission.

Delegation and Validation

I used ChatGPT to research database options, compare their benefits and costs, and review my RFC for errors/imporovements ensuring it met rubric requirements. We are still planning the schema, so no database tests have been completed. Our team uses GitHub to manage code changes and reviews. We use pull requests to review database changes before merging them. Any suggestion provided by AI tools is fact-checked for accuracy.

Decision

Proposed

I recommend Option A for YardWise because planning the tables and relationships before development will help prevent potential future difficulties. It will also ensure the whole team is on the same page.

References

[1]“Chapter 11. Indexes,” PostgreSQL Documentation, Aug. 08, 2024. https://www.postgresql.org/docs/current/indexes.html (accessed Oct. 25, 2026).

[2] Supabase, “Row Level Security | Supabase Docs,” supabase, Oct. 13, 2026. https://supabase.com/docs/guides/database/postgres/row-level-security (accessed Oct. 25, 2026).